

Implementation-Grade Survey for Hypothesis-Tree Derivation Search in MATHDEVMCP

2026-07-07

Abstract

MATHDEVMCP needs a stronger engine for hard mathematical document repair. The current workflows can locate gaps, discover some missing assumptions, and abstain safely, but difficult repairs often require a search over alternative formalizations, assumption sets, derivation decompositions, and backend routes. This note surveys the relevant theorem-proving and language-agent literature, inspects official code where available, and turns the findings into an implementation decision document. The recommendation is to build a *MathDevMCP-native evidence-grounded hypothesis tree*: use deterministic source parsers and backends as the environment, borrow best-first/MCTS-style branching and proof-state data structures from existing systems, and keep Lean/Pantograph/LeanDojo integrations optional adapters rather than required core dependencies.

Contents

1	Purpose, Evidence Contract, and Non-Claims	3
2	Local Baseline and the Actual Problem	3
2.1	Current MATHDEVMCP Behavior	3
2.2	The Mathematical Search Problem	4
2.3	Why the Current Reports Regress	5
2.4	Minimum Agent-Consumable Output	5
2.5	Example: Finite-State Conditional Expectation Branch	6
3	Code Inspection Protocol	6
4	Code and Paper Evidence Ledger	7
4.1	Local Paper Copies	7
4.2	Clone Manifest	7
4.3	Local Probes	7
4.4	Readiness Classification	8
4.5	Candidate-by-Candidate Result	8
5	Second-Pass Citation Snowball and Borrowability Audit	10
5.1	Scope and Limitation	10
5.2	What the Snowball Pass Added	10
5.3	Coverage Matrix for Readily Borrowable Mechanisms	11
5.4	Most Readily Borrowable Ideas	12
5.5	Immediate Design Delta	13

6	Algorithm Families	13
6.1	Sledgehammer, CoqHammer, and the Hammer Pattern	13
6.2	Aesop: Best-First Rule Search in Lean	14
6.3	LeanDojo, ReProver, and LeanDojo-v2	15
6.4	PyPantograph	17
6.5	CoqGym, HOList, GamePad, and TacticZero	18
6.6	HyperTree Proof Search and Evariste	18
6.7	Draft-Sketch-Prove, Thor, and Baldur	19
6.8	Tree of Thoughts	19
6.9	Language Agent Tree Search	19
6.10	DeepSeek-Prover, HunyuanProver, and Modern Guided Tree Search	20
7	Decision: What Should MathDevMCP Build?	21
7.1	Do Not Pick a Single External Algorithm as Core	21
7.2	Recommended Core Algorithm: Evidence-Grounded Hypothesis Tree	21
7.3	Optional Adapter Interfaces	23
7.4	Implementation Blueprint	23
7.5	Parallelism and Search Budget	25
7.6	Required Report Sections	25
8	Implementation Readiness Ranking	26
9	Implementation Decision Matrix	26
10	How the Report Should Change	27
11	Proposed Next Implementation Plan	27
11.1	First Experiment After Implementation	28
12	Final Recommendation	28

1 Purpose, Evidence Contract, and Non-Claims

Question. Which existing algorithms and codebases should guide or support a MATHDEVMCP feature that searches over derivation hypotheses and missing-assumption branches, records blocker nodes, and delegates checkable steps to deterministic backends?

Why this matters. The weak report failure is not only a rendering problem. A report line such as “justify the reconstructed equality from context” is weak because the system has not searched for the missing derivation. For hard documents, the missing piece may be an assumption, a definition, a source-span reconstruction, a rewrite route, a measure-theoretic condition, a shape/type condition, or a formalization choice. A single linear attempt is structurally underpowered.

Primary criterion. This document should be self-contained enough for an implementation decision. For each major candidate it records:

- the problem the paper/tool solves;
- the actual state/action/search/scoring algorithm;
- what code exists and what was inspected locally;
- whether the code appears directly installable or usable;
- whether MATHDEVMCP should reuse it, integrate against it, adapt the algorithm only, or avoid it.

Veto diagnostics. This note must not treat LLM output as proof, must not claim installability without a local check, must not recommend direct reuse without inspecting dependencies and runtime shape, and must not confuse proof-assistant theorem proving with mathematical-document repair.

Not concluded. This is not a benchmark. It does not prove that any external code can solve the credit-card NPV or risky-debt documents. It is a decision artifact for the next implementation plan.

2 Local Baseline and the Actual Problem

2.1 Current MATHDEVMCP Behavior

Local inspection shows these relevant workflows:

- **derive_or_refute:** split target into lhs/rhs, discover assumptions, route to a backend, optionally run counterexample search, and return one result.
- **derive_from:** wrap the low-level derivation/refutation result into a high-level result with gaps, proposals, tool uses, and non-claims.
- **assumptions_for:** propose route-required assumptions and possible sufficient assumption sets.
- **audit_and_propose_fix:** audit labels and use `ProcessPoolExecutor` across labels.

- `audit_math_document_rigor`: render a document-level report with concrete-repair and diagnostic-abstention ledgers.

The repo has parallelism across labels. It does not yet have a within-problem search tree that expands alternative assumption branches, subgoals, and backend routes for one hard derivation.

2.2 The Mathematical Search Problem

For a hard document target, the system receives a claim T , source context C , and known assumptions A_0 . It wants to produce one of:

1. a derivation of T from source-backed assumptions and checked steps;
2. a refutation/counterexample;
3. a best-progress branch with exact remaining blockers;
4. a budget-exhausted search tree that is still useful to a human or agent.

The search state is not simply a Lean proof state. For MATHDEVMCP it should include:

$$s = (T, C, A, G, D, B, E, P)$$

where A is the current assumption set, G is the list of subgoals, D is the partial derivation, B is the blocker ledger, E is backend evidence, and P is the candidate patch/source-span state.

Actions include:

- add a candidate assumption set;
- split the target into subgoals;
- recover or expand a source span;
- rewrite an expression using a definition or accounting identity;
- run a backend check;
- ask a proof assistant for tactic/proof-state feedback;
- mark a branch blocked with evidence.

The success condition is not “the agent sounds confident.” It is:

Every step on a winning branch is either backend checked, source-backed and accepted as an explicit assumption, or explicitly marked as an unchecked modeling convention.

2.3 Why the Current Reports Regress

The weak-report regression has a structural cause. The current high-level reports can say that a problem is unresolved, and they can sometimes produce a local assumption proposal, but they do not yet maintain a search state across multiple attempted repairs for the same target. When a branch fails, the failure is summarized rather than converted into a new, typed subproblem. This causes three concrete pathologies:

1. **One-route collapse.** A target is sent through one derivation route. If that route is inconclusive, the report falls back to generic advice even when another route is available.
2. **Missing blocker memory.** A failure such as “conditional law not defined” is not retained as a node that can be expanded by postulating a finite-state kernel, a measurable transition kernel, or a deterministic sequence convention.
3. **No branch-level evidence accounting.** The renderer cannot say which assumptions made which derivation step work, because the underlying result is not a branch containing assumptions, checked steps, rejected alternatives, and remaining blockers.

The tool that is missing is therefore not only “a better renderer.” The missing tool is a reusable search controller whose output is already shaped as an agent-consumable repair report.

2.4 Minimum Agent-Consumable Output

For a hard target, the function should produce a result with this contract:

```
{
  "question": "can we derive target T from context C?",
  "budget": {"max_nodes": ..., "max_depth": ..., "timeouts": ...},
  "best_branch": {
    "status": "proved|refuted|partial|blocked|budget_exhausted",
    "assumptions_added": [...],
    "derivation_steps": [
      {
        "claim": "...",
        "justification": "...",
        "checker": "parser|sympy|sage|lean|source_assumption",
        "checker_status": "certified|accepted_assumption|not_certified",
        "evidence_ref": "..."
      }
    ],
    "patch": {"assumption_latex": "...", "replacement_latex": "..."}
  },
  "blocked_branches": [...],
  "rejected_branches": [...],
  "non_claims": [...]
}
```

The result should be directly consumable by an agent: it should say where the claim is, what was tried, which assumptions were introduced, why those assumptions close a mathematical gap, what backend checked which step, and what precise patch should be made if the branch is selected.

2.5 Example: Finite-State Conditional Expectation Branch

A typical stochastic-economics repair should not say only “assume integrability.” It should show the route. Suppose a document uses

$$E_t[m_{t+1}X_{t+1}]$$

without defining the conditional law. One branch can postulate a finite-state shock space $\Omega = \{\omega_1, \dots, \omega_K\}$ with conditional probabilities $p_k(s_t)$. That branch transforms the expectation into

$$E_t[m_{t+1}X_{t+1}] = \sum_{k=1}^K p_k(s_t) m(s_t, \omega_k) X(s_t, \omega_k).$$

The branch then has concrete obligations:

1. $p_k(s_t) \geq 0$ and $\sum_k p_k(s_t) = 1$, so the conditional law is defined.
2. $m(s_t, \omega_k)X(s_t, \omega_k)$ is finite for every k , so the finite expectation is integrable.
3. The algebraic rewrite from expectation notation to weighted finite sum is a definition expansion, not an economic theorem.
4. Any subsequent equality can be checked by SYMPY/SAGEMATH as an algebraic identity over the finite sum, or blocked with the exact missing identity.

This is the level of repair detail the new search controller must make available to the report layer.

3 Code Inspection Protocol

All cloned code lives under:

```
.localresources/hypothesis_search_survey/code/
```

The inspection used:

- `git ls-remote` to check public repository availability;
- shallow `git clone -depth 1` for official repositories;
- README/license/dependency/API inspection;
- lightweight local probes for installed Python modules and executables;
- no heavy model download, Coq/Isabelle install, Hugging Face download, or broad environment mutation.

Local environment probes found:

- SAGEMATH is available at `/usr/bin/sage`.
- SYMPY imports in active Python.
- LEAN/lake/elan binaries exist, but local version checks timed out in the active diagnostic path.

- `lean_dojo`, `pantograph`, `tot`, `torch`, `transformers`, and `openai` were not importable in the active Python used by the probe.
- Existing MATHDEVMCP code already has LeanDojo readiness/policy wrappers, including the requirement that LeanDojo output must be reconstructed and checked by direct Lean before certification.

4 Code and Paper Evidence Ledger

4.1 Local Paper Copies

The following papers were downloaded locally because they materially affect the implementation decision:

- `.localresources/hypothesis_search_survey/papers/leandojo-2306.15626.pdf`.
- `.localresources/hypothesis_search_survey/papers/pantograph-2410.16429.pdf`.
- `.localresources/hypothesis_search_survey/papers/https-2205.11491.pdf`.
- `.localresources/hypothesis_search_survey/papers/tree-of-thoughts-2305.10601.pdf`.
- `.localresources/hypothesis_search_survey/papers/deepseek-prover-v15-2408.08152.pdf`.

4.2 Clone Manifest

Repository	Local HEAD	Primary inspection target
LeanDojo	7a9f600b250a	README, package metadata
LeanDojo-v2	936ea0dd32ff	README, <code>pyproject.toml</code> , prover tree
ReProver	fd6d99c01e8b	<code>prover/search_tree.py</code> , <code>proof_search.py</code>
PyPantograph	f8aee320ee55	<code>pantograph/server.py</code> , <code>search.py</code>
aesop	24fa6b359974	README, <code>lakefile.toml</code> , <code>lean-toolchain</code>
LeanCopilot	2458f7339df4	README, <code>lakefile.lean</code>
CoqGym	a739d99cdf5b	README, setup/dependency structure
Evariste	ef43511f2d23	README, setup files, license
tree-of-thought-llm	8050e67d0e3a	<code>src/tot/methods/bfs.py</code> , requirements
LanguageAgentTreeSearch	853d81614607	<code>programming/mcts.py</code> , requirements
DeepSeek-Prover-V1.5	2c4ba9119eef	README, <code>configs/RMaxTS.py</code> , requirements
lean-explore	3a52d6b9922f	README, MCP/API docs, search models
LeanSearch-v2	94f4888cbaf9	README, client, retrieval pipeline
jixia	755fde27a9cf	README, Lean static-analysis plugins

4.3 Local Probes

Bounded probes were used to avoid mutating the active environment:

- Active Python can import `SYMPY` but cannot import `lean_dojo`, `lean_dojo_v2`, `pantograph`, `tot`, `torch`, `transformers`, `openai`, or `ray`.
- `python3 -m py_compile` passed for the inspected ReProver search files, PyPantograph search/server files, ToT BFS file, LATS `programming/mcts.py`, and DeepSeek-Prover-V1.5 launch/config files.

- `python3 -m compileall` passed for the inspected `lean-explore`, `LeanSearch-v2`, and `jixia` source trees. This is a syntax/readiness probe only; it is not an install or runtime proof.
- A broad compile of the LATS programming directory failed on vendored `human_eval/execution.py`; therefore only the MCTS controller file is treated as locally syntax-checked.
- `python3 -m mathdevmcp.cli doctor` found SAGEMATH and SYMPY but reported Lean readiness as inconclusive in the active diagnostic path.
- No `pip install`, `lake build`, model download, Docker build, OPAM/Coq setup, or GPU run was attempted.

4.4 Readiness Classification

This survey uses three implementation classifications:

Use directly.

The code is small enough, locally installable or already usable, and matches the MATHDEVMCP contract without forcing heavy model or proof-assistant setup.

Integrate/test against.

The code exposes a useful runtime boundary, but requires an environment gate, optional backend env, model/API token, or proof assistant project before it can be used safely.

Not recommended as dependency.

The code is too heavy, stale, non-running, not proof-specific, license-limited, or mismatched with document-repair. Its algorithm may still be worth adapting.

4.5 Candidate-by-Candidate Result

LeanDojo original.

Classification: not recommended as a new dependency. The README says the original package is deprecated and directs new projects to LeanDojo-v2. It remains important for existing compatibility and for understanding the ReProver paper, but new work should not bind the core search controller to it.

LeanDojo-v2.

Classification: integrate/test against later. It is a complete framework for tracing, training, retrieval, agents, and Pantograph-based proving, but its package metadata requires Python 3.11 and heavy dependencies including PyTorch, Lightning, Transformers, DeepSpeed, TRL, PEFT, Ray, OpenAI, PyGithub, and datasets. It belongs in an isolated backend environment, not the core MATHDEVMCP import path.

ReProver.

Classification: adapt algorithm only now; integrate later only through a LeanDojo lane. The local files are valuable because they show a compact proof-search skeleton: status enum, internal nodes, edges, cumulative priority, best-first queue, tactic execution, status propagation, and proof extraction. Direct reuse pulls in LeanDojo, PyTorch, Ray, vLLM, and model checkpoints, so the immediate value is architectural.

PyPantograph.

Classification: **integrate/test against after an environment gate.** It is the best optional proof-state adapter candidate. Its Python package has no ordinary Python dependencies, but the build script bundles a Lean REPL and requires matching Lean/lake setup. It should implement a `ProofStateBackend` after direct Lean readiness is stable.

Aesop.

Classification: **integrate/test against as a Lean-side tactic after a Lean project gate; adapt algorithm now.** Aesop is not a Python package. Its safe/unsafe rule discipline and best-first AND/OR tree are a strong template for MATHDEVMCP expansion operators.

Lean Copilot.

Classification: **not core; possible later experiment.** It is useful as a Lean-side tactic/model interface reference, but requires Lean project setup, C++/CMake machinery, model assets, and Git LFS.

CoqGym.

Classification: **not recommended as dependency.** It is useful as a replayable proof-state dataset reference. Direct use requires a Coq/SerAPI/CoqHammer/OPAM/Docker stack that does not match the current Lean/Sage lane.

Evariste / HTTPS.

Classification: **not recommended as dependency.** The README says the code does not run out of the box after Meta-internal components were removed, and the majority license is CC-BY-NC. The AND/OR hypergraph algorithm is still important.

Tree of Thoughts.

Classification: **not recommended as dependency.** The code is simple and the BFS file syntax-checks, but the system is prompt/API centric and depends on the old OpenAI package. Borrow only the generate/evaluate/select abstraction.

Language Agent Tree Search.

Classification: **not recommended as dependency.** The MCTS file syntax-checks and is a good control-loop reference, but the broader programming tree has a syntax error in vendored HumanEval code and the project depends on API/task executors. Borrow the feedback/backprop ledger idea.

DeepSeek-Prover-V1.5.

Classification: **not core; strategic reference and possible future high-budget benchmark.** The repo is public and the inspected config exposes RMaxTS with large settings: batch size 512, Lean concurrency 64, sample count 6400, and 256 search processes. Requirements include PyTorch, Transformers, vLLM, FlashAttention, and GPU-oriented inference. It is evidence that hard proof search benefits from verifier feedback and many attempts, not a drop-in document-repair component.

HunyuanProver.

Classification: **paper/strategy reference only for this survey.** A GitHub repository search for HunyuanProver returned no obvious official repo in the bounded probe. Do not plan an integration until an official codebase is identified and inspected.

LeanExplore.

Classification: **readily borrowable integration pattern; possible optional dependency after environment review.** The cloned repo is a Python package with Apache-2.0 license, a

hosted API client, a local hybrid retrieval backend, and an MCP server. Its MCP design is especially relevant: agents first call a low-token `search_summary` tool and then fetch only the source, docstring, module, dependencies, or description for the few declarations they need [18]. This is directly portable to MATHDEVMCP.

LeanSearch-v2.

Classification: borrow algorithms and client shape now; do not make the local server a core dependency. The repo targets global premise retrieval with a standard retrieval endpoint and a reasoning mode. Its local standard pipeline requires GPU-backed cuVS, Qwen embedding and reranking models, and Python 3.11; its thin HTTP client is lightweight. The most useful immediate ideas are global premise retrieval, decompose/retrieve/ filter/judge loops, and the rule that proof attempts must use only retrieved tools/theorems [19].

jixia.

Classification: borrow source-analysis contract now; optional Lean adapter later. jixia is a Lean 4 static-analysis tool that emits JSON for declarations, symbols, elaboration information, proof-state lines, and ASTs. Its README stresses exact Lean-version matching. This makes it a strong template for source-range and dependency ledgers, but not a zero-setup core dependency [20].

5 Second-Pass Citation Snowball and Borrowability Audit

5.1 Scope and Limitation

After the first survey pass, a citation-snowball script was written under `.localresources/hypothesis_search_survey/snowball/` to query Semantic Scholar for references and citations of the seed papers. The API returned HTTP 429 for all seed papers in the bounded run, and `candidate_papers.json` is therefore empty. This note must not claim a complete forward/backward citation graph.

The fallback pass used three evidence sources:

1. local PDF text mining from LeanDojo/ReProver, Pantograph, HTPS, Tree-of-Thoughts, and DeepSeek-Prover-V1.5;
2. targeted arXiv/GitHub searches for systems named in those related-work and comparison sections;
3. shallow local clone inspection for the most directly borrowable code surfaces.

This is a bounded implementation survey, not an exhaustive bibliometric review. Its purpose is to decide which mechanisms MATHDEVMCP can borrow soon.

5.2 What the Snowball Pass Added

The first pass already covered major proof-search families, but it underweighted premise retrieval and declaration search. The second pass surfaced four borrowable clusters:

1. **Agent-token-efficient declaration search.** LeanExplore exposes a low-token MCP workflow: search summaries first, then per-field source, docstring, dependency, or module fetches. This matches MATHDEVMCP’s goal of reducing agent hallucination and token use.

2. **Global premise retrieval.** LeanSearch-v2 and ReProver both show that hard proof search needs a premise index before generation. LeanSearch-v2 is too heavy as a core dependency, but its standard-client boundary and reasoning retrieval loop are reusable design patterns.
3. **Source-range and dependency extraction.** jixia, LeanDojo, and CoqGym all treat source ranges, dependencies, proof states, and replay logs as first-class data. MATHDEVMCP should do the same for LaTeX labels, assumptions, equations, and derived obligations.
4. **Partial-progress proof search.** DeepSeek-Prover-V1.5’s truncate-and-resume search and RMaxTS exploration mechanism reinforce the blocker-node design: a failed attempt should become a reusable partial path, not a generic report sentence.

5.3 Coverage Matrix for Readily Borrowable Mechanisms

Family or system	Mechanism	Borrowability for MATH-DEVMCP	Concrete action
LeanExplore	MCP/API search with <code>search_summary</code> plus per-field getters for source, dependencies, docstrings, descriptions, and modules	High. The interaction pattern is directly agent-consumable and token-frugal; local backend is optional	Add a summary-first context-search provider with field-fetch methods; later add optional LeanExplore adapter
LeanSearch-v2	Global premise retrieval; standard HTTP client; reasoning mode that decomposes, retrieves, filters, and judges candidate premises	Medium. The client shape and reasoning loop are useful; the full local server is GPU and model heavy	Borrow the decompose–retrieve–filter–judge contract and “use only retrieved tools” rule; keep GPU server outside core
jixia	Non-intrusive Lean source analysis: declarations, symbols, elaboration, line proof states, AST dumps	Medium. Great contract for exact source provenance; requires exact Lean-version matching	Mirror the JSON evidence contract for LaTeX source spans now; make jixia a future optional Lean static analysis backend
ReProver/LeanDojo	Accessible-premise restriction, DPR-style premise retrieval, best-first tactic search, status propagation	High for algorithm, low for direct dependency	Build a document-level premise index restricted to nearby/source-accessible assumptions before hypothesis expansion
Aesop	Normalize/safe/unsafe rule phases; best-first search over goals	High. This is a simple discipline to encode in Python	Require every expansion provider to declare <code>normalize</code> , <code>safe</code> , or <code>unsafe</code> ; unsafe operators must branch and record assumptions
Pantograph	Machine-to-machine Lean proof-state API; MCTS-compatible search state; goal/tactic branching	Medium after Lean environment gate	Define <code>ProofStateBackend</code> ; use Pantograph for search evidence but certify with direct Lean

Family or system	Mechanism	Borrowability for MATH-DEVMCP	Concrete action
Hammer family	Select premises, translate to ATP, search, reconstruct proof in the assistant	High as proof discipline, not as Python dependency	Keep external search separate from certification; every repair must reconstruct through a deterministic backend or explicit assumption
HTTPS/Evariste	AND/OR hypergraph for tactics that create multiple subgoals	High for data model, low for dependency	Add <code>AlternativeNode</code> and <code>ObligationSetNode</code> ; conjunctive assumption obligations must all close
DeepSeek-Prover-V1.5/RMaxTS	Truncate-and-resume MCTS, intrinsic reward for new proof states, root/tree parallelization	Medium for future deep profile	Implement smoke/standard/deep budget ladder; in deep mode reward new checked states and keep partial paths
Thor/Baldur/DSP	Generate candidate proof, run checker, repair from verifier feedback, accept only checked scripts	High as report-promotion policy	Add regression tests forbidding concrete fixes without checker evidence or accepted assumptions
Proof-state datasets	Replayable proof-state environment and dataset format	Medium as logging format, low as dependency	Serialize every branch, backend attempt, source span, and blocker to replayable JSON

5.4 Most Readily Borrowable Ideas

The highest-return borrowable ideas are not large prover models. They are small contracts that make agents less speculative.

1. Summary-first retrieval. Borrow LeanExplore’s MCP design. A MathDevMCP agent should not receive an entire document index or full Lean source dump by default. It should call:

```
search_context_summary(query, limit, scope)
get_context_source(context_id)
get_context_assumptions(context_id)
get_context_dependencies(context_id)
get_context_location(context_id)
```

This is immediately useful for LaTeX documents even before Lean integration: the same API can search labels, equations, assumptions, definitions, theorem statements, and nearby prose.

2. Accessible-premise filtering. Borrow ReProver’s accessible-premise idea. For document repair, the accessible premise set should include assumptions and definitions in the same theorem environment, earlier equations in the same section, declared global assumptions, imported notation

conventions, and user-provided assumptions. It should exclude later claims unless the document explicitly permits forward references. This prevents the agent from proving a step using facts the text has not established.

3. Decompose/retrieve/filter/judge before generation. Borrow LeanSearch-v2’s reasoning pattern but replace free-form judging with structured evidence. For a hard target:

1. decompose the target into needed mathematical objects and subclaims;
2. retrieve candidate assumptions, definitions, lemmas, and backend routes;
3. filter by source accessibility and type/shape compatibility;
4. judge only whether the retrieved material can support the next checked obligation.

Only after this should a branch synthesize a repair patch.

4. Source-range evidence as a first-class object. Borrow jixia/CoqGym’s source and replay mindset. A repair proposal must carry the exact file, label, line, source span, dependency IDs, and checker attempts. This should become a reusable **EvidenceRef** object rather than prose in a report.

5. Partial-progress tree nodes. Borrow DeepSeek-Prover-V1.5’s truncate-and-resume lesson. If a branch derives steps 1–3 and fails at step 4, the tree should store the successful prefix and expand from the blocker. It should not collapse the whole attempt into “collect more evidence.”

5.5 Immediate Design Delta

The next implementation plan should add three small, reusable components before any heavyweight prover integration:

1. **ContextSearchProvider**: summary-first retrieval over document labels, equations, assumptions, definitions, and optional Lean declarations.
2. **AccessiblePremiseIndex**: deterministic source-accessibility and dependency filtering for each target label or source span.
3. **EvidenceRef**: stable references for source span, retrieved premise, backend attempt, blocker, and accepted assumption.

These components are the easiest borrow from the literature and code search, and they directly attack the regression that produced hand-wavy reports.

6 Algorithm Families

6.1 Sledgehammer, CoqHammer, and the Hammer Pattern

Problem solved. Interactive proof assistants have rich libraries but users do not want to find all premises and write all low-level proof steps manually. Hammers bridge interactive proof assistants with external automated provers: select relevant facts, translate a goal, run automated provers, and reconstruct a checked proof inside the proof assistant when possible [1, 2, 3].

Algorithm sketch.

1. Input a typed proof-assistant goal and local context.
2. Select relevant premises from the library.
3. Translate the goal and premises to an ATP/SMT-friendly representation.
4. Run one or more automated provers under timeouts.
5. If a proof is found, reconstruct a proof script in the assistant.
6. Return either a checked proof/reconstruction or prover diagnostics.

Relevance to MATHDEVMCP. The hammer pattern is the right discipline: search and proposal may happen outside the certifier, but the certifier boundary is explicit. For MATHDEVMCP this means the hypothesis tree can propose source-backed assumptions and subgoals, but final proof claims must be checked by Lean or a scoped deterministic backend.

Implementation lesson. Premise/assumption selection is a first-class component, not an afterthought. MATHDEVMCP should retrieve definitions, assumptions, and local equations from the target document before expanding speculative branches.

Translation to document repair. For MATHDEVMCP, the hammer pattern should become:

1. Parse the target and collect local definitions, assumptions, labels, and nearby equations.
2. Select a small premise set: source-backed assumptions first, then deterministic route-required assumptions, then optional modeling conventions.
3. Translate the target to the strongest backend-supported form: finite algebra for SYMPY/SAGEMATH, direct Lean statement for local formal goals, or typed diagnostic obligation if no backend can encode it.
4. Run multiple backend attempts under timeouts.
5. Promote only reconstructed/checkable results; otherwise return the exact missing premise or failed translation as a blocker.

6.2 Aesop: Best-First Rule Search in Lean

Problem solved. Aesop automates “obvious” Lean proofs by applying a configurable set of rules. It is useful both as general automation and as a framework for domain-specific automation [4].

Algorithm. Aesop builds a search tree of Lean goals. Rules can be safe or unsafe. Safe rules are applied eagerly and are not backtracked; unsafe rules are assigned a success probability. The tree is explored best-first: goals reached through high-probability rules are prioritized. Before rule application, goals are normalized with configurable normalization rules, including `simp_all`.

Important mechanics.

- State: Lean goal state.
- Action: apply a rule/tactic.
- Safe action: deterministic progress; no backtracking.
- Unsafe action: branch with a probability/priority.
- Scoring: multiply parent priority by rule success probability.
- Termination: proof found, rule space exhausted, or resource limits.
- Debugging: trace options expose search steps and tree.

Code readiness. The Aesop repo was cloned. It is a Lean package with `lakefile.toml` and `lean-toolchain`. It is not a Python library. Direct use requires a matching Lean project and lake build.

Recommendation. Use Aesop as both an algorithmic template and a Lean-side tactic. The MATHDEVMCP hypothesis tree should copy the safe/unsafe distinction: parser-normalization, complete source-span recovery, and dimension checking are safe deterministic actions; adding an economic convention or general integrability condition is unsafe and should branch.

Direct design requirement. Every MATHDEVMCP expansion operator should declare one of three phases: `normalize`, `safe`, or `unsafe`. A `normalize` operator must not introduce a new assumption; it may only canonicalize notation, recover a source span, normalize dimensions, or unfold definitions. A `safe` operator must be provability-preserving under the current branch assumptions. An `unsafe` operator may introduce a modeling convention or one of several sufficient assumption sets, and therefore must create a branch with an evidence-labeled hypothesis. This one rule would prevent many hand-wavy reports: an operator that cannot justify its safety must be recorded as a branching hypothesis, not as a fix.

6.3 LeanDojo, ReProver, and LeanDojo-v2

Problem solved. LeanDojo provides programmatic interaction with Lean and extraction of proof states/tactics/premises from Lean repositories. ReProver adds retrieval-augmented tactic generation: retrieve relevant premises and generate tactics conditioned on the proof state and retrieved premises [9, 10]. LeanDojo-v2 expands this into an end-to-end framework with repository tracing, agents, trainers, Pantograph-based provers, and external APIs.

ReProver best-first algorithm. The inspected `prover/search_tree.py` and `prover/proof_search.py` implement:

- Node statuses: `PROVED`, `FAILED`, `OPEN`.
- Internal node state: Lean tactic state and cumulative action log probability.
- Edge: a tactic from a source state to a destination state.

- Priority: cumulative log probability.
- Expansion: pop highest-priority open node; generate tactic suggestions; run each tactic in Lean-Dojo; create proof-finished, error, or new internal nodes.
- Status propagation: a node is proved if any child is proved; failed if all children failed.
- Proof extraction: recursively choose child edge with shortest distance-to-proof.
- Stopping: proof found, root failed, priority queue empty, timeout, or max expansions.

What to borrow. This is a nearly perfect skeleton for a MATHDEVMCP search controller, but the state should be broader than Lean tactic state. Replace tactic generation with expansion operators such as assumption expansion, source-span expansion, subgoal split, backend attempt, and patch synthesis. Replace cumulative tactic logprob with evidence-grounded branch priority.

Implementation skeleton to adapt. The MATHDEVMCP controller can adapt the ReProver loop almost literally:

```
root = SearchNode(target=T, assumptions=A0, status="open", score=0)
queue = priority_queue([root])
while queue and not budget.exhausted():
    node = queue.pop_best()
    expansions = expansion_policy(node)
    edges = []
    for expansion in expansions:
        child = apply_expansion(node, expansion)
        child = run_available_backends(child)
        child.status = propagate_local_status(child)
        edges.append(SearchEdge(expansion, child))
        if child.status in {"open", "partial"}:
            queue.push(child)
    node.out_edges = edges
    propagate_status_to_ancestors(node)
return extract_best_branch(root), render_blockers(root)
```

The differences from ReProver are the important part. The action is not a Lean tactic; it is a repair operator. The environment is not only Lean; it is the set of deterministic MATHDEVMCP tools. The reward is not a model log probability; it is evidence quality and patchability.

Code readiness. The original LeanDojo clone is marked deprecated by its README. LeanDojo-v2 is more complete but heavy: it requires Python 3.11, PyTorch, Transformers, DeepSpeed, TRL, PEFT, Ray, GitHub/Hugging Face tokens for common workflows, and Pantograph for Lean RPC. Active MATHDEVMCP Python did not import `lean_dojo` or `lean_dojo_v2` during the probe.

Recommendation. Do not make LeanDojo-v2 a required dependency of MATHDEVMCP core. Implement a native search controller and define an optional LeanDojo/Pantograph adapter that can be enabled in a backend environment. Reuse the search-tree structure and proof-attempt ledger ideas.

6.4 PyPantograph

Problem solved. Pantograph exposes Lean 4 as a machine-to-machine interaction environment: start goals, execute tactics, inspect states, load sketches, check files, and manage proof-state branching [11].

Algorithm/API mechanics. The inspected `pantograph/server.py` exposes a `Server` with methods such as:

- `goal_start`: create a goal state from an expression;
- `goal_tactic`: run a tactic on a selected goal;
- `goal_continue`: resume a target after branch search;
- `expr_type`: check expression types;
- error handling for parse errors, tactic failure, unsafe/sorry-producing tactics.

The inspected `pantograph/search.py` provides:

- a `SearchState` with goal state, parent, goal priorities, children, tried tactics, visit counts, and exhaustion flags;
- an `Agent` interface with `next_tactic` and `guidance`;
- an `MCTSAgent` interface with `estimate`, `select`, and `backup`.

Code readiness. PyPantograph was cloned. It is small and clean, with Apache-2.0 license and a Python package. But building requires its Lean REPL with matching Lean version: the build script runs `lake build repl` and copies the compiled REPL into the Python package. Active Python did not import `pantograph`; no build was attempted.

Recommendation. This is the best direct integration candidate after an environment gate. For the next MATHDEVMCP phase, do not block on Pantograph. Design a `ProofStateBackend` interface so Pantograph can later implement `start_goal`, `run_tactic`, `check_expr`, and `branch`.

Concrete adapter boundary. The Pantograph adapter should not be allowed to certify a document repair by itself. It should return proof-state evidence:

```
{
  "backend": "pantograph",
  "goal": "...",
  "attempted_tactics": [...],
  "result": "closed|open|tactic_failure|server_error",
  "proof_script_candidate": "...",
  "requires_direct_lean_check": true
}
```

If the branch claims a proof, MATHDEVMCP should reconstruct a standalone Lean theorem and run direct Lean checking. Pantograph is an interactive search environment; direct Lean is the certification boundary.

6.5 CoqGym, HOLLIST, GamePad, and TacticZero

Problem solved. These systems expose theorem proving as a machine-learning environment: state, action/tactic, transition, reward, and datasets of proof states [5, 6, 7, 8].

CoqGym mechanics. CoqGym extracts Coq proofs into JSON/LMDB records containing source commands, proof steps, goals, hypotheses, environments, and tactic commands. This makes proof search learnable and replayable. It also includes an interaction environment and ASTactic model.

Code readiness. The CoqGym repo was cloned and is large even shallow. README explicitly says native setup is nontrivial and recommends Docker. Native setup requires OPAM, OCaml switch, Coq, SerAPI, CoqHammer, Ruby, LMDB, and building Coq projects. No install was attempted.

Recommendation. Use as a data-format reference: MATHDEVMCP should log search nodes and backend transitions in a replayable JSON format. Do not integrate CoqGym code for the hypothesis-search lane.

6.6 HyperTree Proof Search and Evariste

Problem solved. HyperTree Proof Search (HTPS) addresses neural theorem proving where actions can create multiple subgoals. A proof search node is naturally AND/OR-like: a tactic may produce several child goals, and the parent is solved only if all children close [12].

Algorithm. HTPS represents proof search as a hypergraph:

- OR nodes represent goals that may be solved by one of several tactics;
- hyperedges represent tactics;
- each hyperedge points to a set of subgoals;
- the search alternates expansion and value/priority estimation;
- status propagates through the hypergraph: a hyperedge succeeds only if all subgoals succeed.

Code readiness. The Evariste repo was cloned. Its README explicitly says the code does not run out of the box because Meta-internal references were removed without fixing generic entry points. Majority license is CC-BY-NC, with some subcomponents under separate licenses.

Recommendation. Do not integrate Evariste code. Borrow the AND/OR hypergraph idea. This matters for MATHDEVMCP because many derivation branches split into multiple obligations: e.g., conditional expectation requires law, measurability, and integrability; Bellman recursion requires state/action set, transition kernel, reward finiteness, and terminal condition.

Direct design requirement. The search tree should not be a plain OR tree. Some expansions produce a set of obligations that must all close. For example, the finite conditional expectation branch succeeds only if the probability law, finiteness, and algebraic rewrite obligations all close. This requires an AND/OR node type: `AlternativeNode` for competing routes and `ObligationSetNode` for conjunctive subgoals.

6.7 Draft-Sketch-Prove, Thor, and Baldur

Problem solved. These systems use language models to propose informal proof sketches or full proofs, then use formal tools for checking and repair [13, 14, 15].

Algorithm pattern.

1. Generate a candidate proof/sketch/script.
2. Run a proof assistant or ATP-backed checker.
3. Parse failures into repair prompts or subgoals.
4. Iterate under a budget.
5. Accept only checked proof scripts; otherwise return diagnostics.

Relevance. The important idea is not “ask the LLM harder.” It is the verifier loop. A MATH-DEVMCP branch may be proposed by an agent, but every branch must be run through parsers, SYMPY/SAGEMATH, shape/type checks, or LEAN before it can be promoted.

6.8 Tree of Thoughts

Problem solved. Tree of Thoughts (ToT) generalizes single-chain prompting into a search over intermediate thought states [22].

Algorithm from inspected code. The official `bfs.py` implements:

1. Maintain candidate partial outputs y .
2. Generate successors with either `sample` or `propose`.
3. Evaluate candidates with `value` or `vote` prompts.
4. Select top candidates greedily or by sampling.
5. Repeat for a fixed number of task steps.
6. Log generated candidates, values, and selected states.

Code readiness. Repo cloned; `compileall` passed for source. It depends on the old `openai==0.27.7` package and API keys for actual runs. Active Python did not have `openai` or `tot` installed.

Recommendation. Do not reuse code directly. Borrow the `generate/evaluate/select` interface, but replace LLM value prompts with deterministic evidence scoring.

6.9 Language Agent Tree Search

Problem solved. LATS combines language-model generation, environment feedback, reflection, and tree search for reasoning/acting tasks [23].

Algorithm from inspected code. The programming `mcts.py` implements:

- Node fields: solution, parent, children, value, visits, reflection, test feedback.
- Selection: recursively choose child by UCT.
- Expansion: generate n new candidate solutions conditioned on prior solution, feedback, and reflection.
- Simulation/evaluation: run tests, compute reward from passed tests and final success.
- Backpropagation: add reward to the child and ancestors.
- Logging: write solutions, feedback, reflection, and pass/fail status.

Code readiness. Repo cloned. It requires OpenAI/API-key paths and task-specific environments such as HotPotQA/WebShop/programming executors. It is not proof-specific.

Recommendation. Borrow the control loop and logging discipline. Do not reuse as a dependency. For MATHDEVMCP, the environment feedback should be backend evidence, not unit tests over generated code.

6.10 DeepSeek-Prover, HunyuanProver, and Modern Guided Tree Search

Problem solved. Recent prover systems combine large theorem-proving models, synthetic data, proof-assistant feedback, reinforcement learning, and guided tree search for LEAN theorem proving [16, 17, 21].

Algorithmic implications. The lesson for MATHDEVMCP is that hard proving is a high-budget search problem. These systems rely on many generated attempts and verifier feedback. They do not imply that a small local tool should train a prover. They do imply that MATHDEVMCP should record many attempted branches and should not expect one pass to solve hard derivations.

Recommendation. Use as strategic evidence that verifier-guided search is the right direction. Do not depend on large prover models in the core workflow.

Code inspection note. The DeepSeek-Prover-V1.5 repo exposes a high-budget RMaxTS configuration: sample count 6400, concurrent sampling 32, Lean concurrent requests 64, batch size 512, and 256 search processes. This is not a local default for MATHDEVMCP. It is a scale warning: genuinely hard derivations may require many branches. The immediate implementation should therefore expose a budget ladder:

- **smoke:** 10–20 nodes, one or two deterministic routes.
- **standard:** 80–200 nodes, multiple assumption branches and backend attempts.
- **deep:** explicit user-approved run with higher node count, process parallelism, and optional Lean/Pantograph adapters.

7 Decision: What Should MathDevMCP Build?

7.1 Do Not Pick a Single External Algorithm as Core

No inspected codebase is a clean drop-in for MATHDEVMCP:

- LeanDojo-v2 is powerful but too heavy for a core dependency.
- PyPantograph is promising but needs a Lean/Pantograph environment gate.
- Aesop is Lean-side, not a Python document-repair engine.
- Evariste explicitly does not run out of the box.
- ToT/LATS are not proof systems and rely on LLM/API feedback.
- CoqGym is a dataset/environment reference, not a direct integration path.

The right plan is a MATHDEVMCP-native controller with optional adapters.

7.2 Recommended Core Algorithm: Evidence-Grounded Hypothesis Tree

State. Each node should record:

```
node_id
parent_id
depth
target
source_span
claim_kind
postulated_assumptions
accepted_assumptions
subgoals
derived_steps
backend_attempts
blockers
patch_candidates
score
status
```

Statuses.

- **proved**: all required obligations are certified or accepted under explicit assumptions.
- **refuted**: a backend/counterexample refuted the target.
- **blocked**: expansion hit a concrete blocker.
- **open**: branch can still be expanded.
- **budget_exhausted**: not solved within search budget.
- **rejected**: malformed or unsupported candidate.

Actions.**Source-span expansion.**

Recover complete equation/proof span. Evidence: LaTeX parser and localization check.

Assumption expansion.

Add finite-state, kernel-integrability, shape, Bellman, accounting, or differentiability branch. Evidence: `assumptions_for`, domain templates, and source references.

Subgoal split.

Turn target into local obligations. Evidence: typed obligation IR.

Algebra/backend check.

Try SYMPY/SAGEMATH on algebraic or finite-sum step. Evidence: backend result.

Lean/Pantograph check.

Try a formal Lean local theorem or tactic path. Evidence: direct Lean final check is required before certification.

Patch synthesis.

Construct exact assumption block and replacement LaTeX. Evidence: parser and source-span checks.

Blocker record.

Stop branch with concrete missing object. Evidence: blocker type and required next artifact.

Scoring. Start with best-first/beam search, inspired by Aesop and ReProver. Use a transparent additive score:

$$\begin{aligned}
 S(n) = & 4C_{\text{LEAN}}(n) + 3C_{\text{SAGEMATH/SYMPY}}(n) + 2C_{\text{parser/type}}(n) \\
 & + 2R_{\text{source}}(n) + P_{\text{patch}}(n) - 3B_{\text{fatal}}(n) \\
 & - 2H_{\text{unsupported}}(n) - M_{\text{malformed}}(n).
 \end{aligned}$$

Later add UCT:

$$UCT(n) = \bar{r}(n) + c\sqrt{\frac{\log N(\text{parent}(n))}{N(n)}}.$$

Here reward $\bar{r}(n)$ must be evidence-grounded: checked steps, fewer missing assumptions, source-backed definitions, and patchability. It must not be LLM confidence.

Expansion policy. For the first implementation, avoid open-ended MCTS. Use:

1. deterministic root normalization;
2. generate a small fixed set of expansion operators;
3. best-first or beam search with `max_nodes`, `max_depth`, `max_branches_per_blocker`, and backend timeouts;
4. process-level parallelism only for side-effect-free backend attempts or branch scoring;
5. stable final ordering for reproducible reports.

7.3 Optional Adapter Interfaces

Define adapters but keep core independent:

```
class SearchBackend:
    def can_handle(obligation) -> BackendCapability
    def attempt(node, timeout) -> BackendAttempt

class ProofStateBackend(SearchBackend):
    def start_goal(statement) -> ProofState
    def run_tactic(state, tactic) -> ProofStateResult

class ExpansionProvider:
    def expand(node, budget) -> list[Expansion]
```

Concrete adapters:

- SymPyBackend: active now.
- SageBackend: active if `/usr/bin/sage` path is usable.
- LeanDirectBackend: direct Lean check only; certification authority.
- PantographBackend: optional proof-state/tactic interaction.
- LeanDojoV2Backend: optional tracing/retrieval/prover backend in isolated environment.

7.4 Implementation Blueprint

Proposed Python entry point. The reusable function should be a pure-Python high-level workflow:

```
hypothesis_search_derivation(
    tex_path: str | None,
    label: str | None,
    target: str,
    context: str | None = None,
    provided_assumptions: list[str] | None = None,
    search_profile: Literal["smoke", "standard", "deep"] = "standard",
    backends: list[str] = ["sympy", "sage", "lean"],
    output_path: str | None = None,
) -> dict[str, Any]
```

It should also have a Markdown renderer:

```
render_hypothesis_search_report(result) -> str
```

The output path is optional. If provided, the function writes a report and returns the same structured JSON result. This matches the successful `audit_math_document_rigor` pattern but gives the agent a stronger within-problem search artifact.

Core data structures.

```
@dataclass(frozen=True)
class SearchBudget:
    max_nodes: int
    max_depth: int
    max_branches_per_node: int
    backend_timeout_s: float
    workers: int

@dataclass
class SearchNode:
    node_id: str
    parent_id: str | None
    kind: Literal["alternative", "obligation_set", "leaf"]
    target: str
    assumptions: list[Assumption]
    obligations: list[Obligation]
    derivation_steps: list[DerivationStep]
    backend_attempts: list[BackendAttempt]
    blockers: list[Blocker]
    patch_candidates: list[PatchCandidate]
    score: float
    status: Literal[
        "proved", "refuted", "partial", "blocked",
        "open", "budget_exhausted", "rejected"
    ]
```

Expansion providers.

Source span provider. Takes a label or line range and returns the smallest complete equation/proof span. It must record file, line, section path, and parser confidence.

Assumption provider. Calls or generalizes `assumptions_for` to produce alternative sufficient assumption sets, each with mathematical rationale and source/provenance.

Derivation split provider. Converts one large equality or prose claim into typed obligations: definition expansion, algebraic equality, shape/type constraint, stochastic expectation condition, or modeling convention.

Backend provider. Runs routeable obligations through SYMPY, SAGEMATH, direct Lean, or optional Pantograph/LeanDojo adapters.

Patch provider. Produces exact assumption blocks and replacement LaTeX only when all required branch steps are either checked or explicit accepted assumptions.

Promotion rule. A branch may be reported as a concrete repair only if:

1. every derivation step has a checker status of `certified` or `accepted_assumption`;

2. every accepted assumption has a location, statement, why-needed field, and derivation role;
3. every backend claim has a backend attempt record;
4. every patch has the source span it replaces and the assumptions it introduces;
5. all remaining gaps are explicitly in the diagnostic-abstention ledger.

7.5 Parallelism and Search Budget

The literature supports parallel search, but the first implementation should use controlled process-level parallelism only where branch attempts are side-effect-free. A good initial policy is:

- expand the priority queue deterministically in the parent process;
- submit independent backend attempts for sibling branches to a process pool;
- merge results by stable node id, not by completion order;
- cap backend timeouts separately from global search budget;
- record every timeout as a blocker or rejected backend attempt, not as a mathematical failure.

This gives speed without making reports non-reproducible.

7.6 Required Report Sections

The Markdown/LaTeX report should include:

1. **Question and location.** The target, source file, line range, and section path.
2. **Budget and tools.** Search profile, node/depth/backend timeouts, and exact backend/tool calls.
3. **Best branch.** Assumptions added, derivation steps, backend evidence, and patch candidates.
4. **Why assumptions are missing.** For each added assumption, show the failed obligation it closes.
5. **How derivation works under assumptions.** Step-by-step route with checked or accepted evidence status.
6. **Blocked alternatives.** Branches attempted, where they stopped, and the smallest artifact needed to continue.
7. **Non-claims.** No proof of global minimality, no claim about unsearched branches, no certification from LLM-only text, and no Lean proof unless direct Lean passed.

8 Implementation Readiness Ranking

1. **Implement now: native search controller.** The core can be pure Python using existing MATHDEVMCP structures and deterministic backends.
2. **Implement now: deterministic expansion templates.** Start with conditional expectation, finite-horizon NPV, accounting identity, Bellman recursion, and shape/dimension constraints.
3. **Implement next: optional Pantograph adapter.** PyPantograph is the cleanest proof-state API but requires environment setup.
4. **Implement next: Lean direct final-check discipline.** Every Lean-derived branch must end in direct Lean checking.
5. **Defer: LeanDojo-v2/ReProver model integration.** Useful but heavy; needs separate back-end env, tokens, model dependencies, and benchmark plan.
6. **Reference only: Evariste/HTPS code, CoqGym, ToT, LATS.** Borrow algorithms/data formats/control loops, not runtime dependencies.

9 Implementation Decision Matrix

Native evidence-grounded search controller.

Expected value: directly fixes weak reports and is compatible with current tools. Main risk: schema and tests must be disciplined. Decision: start here.

Pantograph optional adapter.

Expected value: Lean proof-state feedback and tactic search. Main risk: Lean/lake/Pantograph environment gate. Decision: build after the core controller.

LeanDojo-v2 / ReProver integration.

Expected value: retrieval and trained prover infrastructure. Main risk: heavy dependencies, tokens, and model/runtime cost. Decision: separate backend lane only.

Aesop Lean-side tactic.

Expected value: strong automation for generated Lean goals. Main risk: Lean project and domain formalization required. Decision: use when a direct Lean route exists.

DeepSeek-Prover-V1.5 style search.

Expected value: evidence for high-budget verifier-guided search. Main risk: GPU/model/proof benchmark cost. Decision: strategic reference, not core.

ToT/LATS code reuse.

Expected value: simple tree-search control examples. Main risk: LLM/API-centric and not proof-specific. Decision: borrow abstractions only.

CoqGym/Evariste code reuse.

Expected value: rich proof-search history. Main risk: heavy, non-running, or mismatched stack. Decision: do not integrate.

The decision is not to pick a paper implementation wholesale. The decision is to build the missing MATHDEVMCP abstraction that all successful systems share: explicit search state, verifier feedback, status propagation, budgeted branching, and replayable evidence.

10 How the Report Should Change

A derivation-grade report should no longer say:

Derivation route: justify the reconstructed equality from context.

It should say:

Search budget: 80 nodes, depth 5, 4 workers. Best branch: finite-state conditional expectation. Source span was complete and parsed. Step 1 expands the NPV definition; parser checked. Step 2 applies linearity of finite conditional expectation; SYMPY checked the finite weighted-sum algebra. Step 3 uses the economic zero-profit pricing convention; this remains an explicit modeling assumption. Blocked alternatives: kernel-integrability branch lacked a conditional kernel; Bellman branch lacked terminal boundary. Patch: insert Assumption A and replace equation block B.

This is the practical standard for implementation.

11 Proposed Next Implementation Plan

1. Add `hypothesis_search.py` with node/edge/budget dataclasses, status propagation, stable best-first queue, JSON serialization, and deterministic scoring.
2. Add source-span and assumption expansion providers. The first target templates should be finite conditional expectation, finite-horizon NPV, accounting identity, Bellman recursion, and shape/dimension constraints.
3. Add backend attempt provider that reuses the existing route layer for SYMPY/SAGEMATH/direct Lean where available. Backend unavailable must become a blocker, not a refutation.
4. Add patch synthesis only after a branch satisfies the promotion rule. Patch records must include replacement span, inserted assumptions, and derivation role for each assumption.
5. Add report renderer with required sections: best branch, derivation under assumptions, blocked alternatives, exact tools used, budget, and non-claims.
6. Add tests that forbid concrete repairs without checked steps or accepted assumptions. Include regression tests for the previous hand-wavy phrases such as “justify from context” and “collect more evidence”.
7. Integrate into `audit_math_document_rigor` for selected hard labels behind a profile flag. Keep the old path as fallback until reports are stable.
8. Add optional Pantograph adapter only after the core controller passes tests and direct Lean readiness is reliable in a separate backend env.

11.1 First Experiment After Implementation

The first application experiment should be the credit-card NPV document because it contains exactly the kind of finite-state, accounting, discounting, and expectation assumptions that stress the search design. The evidence contract should be:

Question. Can the new search controller produce concrete, derivation grade repairs instead of diagnostic-only comments for selected NPV claims?

Baseline. Current `audit_math_document_rigor` report on the same labels.

Promotion criterion. At least one target produces a best branch with explicit assumptions, a derivation route under those assumptions, checked or accepted evidence per step, and exact patch text.

Veto diagnostics. Any concrete repair lacking location, assumption role, backend/source evidence, or patch span is a failure.

Non-claim. Passing this experiment does not prove the whole document is rigorous; it only validates the search/report machinery on selected targets.

12 Final Recommendation

The implementation should not start by installing a large external prover stack. It should start by making MATHDEVMCP search explicit and reproducible:

agent proposes candidate branches; deterministic tools check nodes; the controller records evidence, blockers, scores, and budgets.

This directly addresses the user’s concern: hard derivation repair is a search problem under uncertainty, not a one-pass report-generation problem.

References

- [1] Lawrence C. Paulson and Jasmin C. Blanchette. Three years of experience with Sledgehammer, a practical link between automatic and interactive theorem provers. In *Proceedings of the 8th International Workshop on the Implementation of Logics*, 2010. <https://isabelle.in.tum.de/doc/sledgehammer.pdf>.
- [2] Jasmin C. Blanchette, Cezary Kaliszyk, Lawrence C. Paulson, and Josef Urban. Hammering towards QED. *Journal of Formalized Reasoning*, 9(1), 2016.
- [3] Lukasz Czajka and Cezary Kaliszyk. Hammer for Coq: Automation for dependent type theory. *Journal of Automated Reasoning*, 61, 2018.
- [4] Jannis Limperg and Lean community contributors. Aesop: Automated proof search for Lean. GitHub repository. <https://github.com/leanprover-community/aesop>.
- [5] Kshitij Bansal, Sarah M. Loos, Markus N. Rabe, Christian Szegedy, and Stewart Wilcox. HOList: An environment for machine learning of higher-order logic theorem proving. In *International Conference on Machine Learning*, 2019.

- [6] Kaiyu Yang and Jia Deng. Learning to prove theorems via interacting with proof assistants. In *International Conference on Machine Learning*, 2019.
- [7] Daniel Huang, Prafulla Dhariwal, Dawn Song, and Ilya Sutskever. GamePad: A learning environment for theorem proving. In *International Conference on Learning Representations*, 2019.
- [8] Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles E. Staats, Mateja Jamnik, and Christian Szegedy. TacticZero: Learning to prove theorems from scratch with deep reinforcement learning. In *NeurIPS*, 2021.
- [9] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J. Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Neural Information Processing Systems Datasets and Benchmarks Track*, 2023. <https://github.com/lean-dojo/LeanDojo>.
- [10] LeanDojo contributors. ReProver: Retrieval-augmented theorem proving for Lean. GitHub repository. <https://github.com/lean-dojo/ReProver>.
- [11] Leni Aniva, Chuyue Sun, Brando Miranda, Clark Barrett, and Sanmi Koyejo. Pantograph: A machine-to-machine interaction interface for advanced theorem proving, high level reasoning, and data extraction in Lean 4. 2024. <https://github.com/stanford-centaur/PyPantograph>.
- [12] Guillaume Lample, Timothée Lacroix, Marie-Anne Lachaux, Thibaut Lavril, Xavier Martinet, Amandine Hayat, Gabriel Ebner, Aurélien Rodriguez, and colleagues. HyperTree Proof Search for neural theorem proving. In *NeurIPS*, 2022.
- [13] Albert Q. Jiang, Wenda Li, Jesse Michael Han, and Yuhuai Wu. Draft, Sketch, and Prove: Guiding formal theorem provers with informal proofs. In *International Conference on Learning Representations*, 2023.
- [14] Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Timothée Lacroix, Jiacheng Liu, Wenda Li, Mateja Jamnik, Guillaume Lample, and Yuhuai Wu. Thor: Wielding hammers to integrate language models and automated theorem provers. In *NeurIPS*, 2022.
- [15] Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. Baldur: Whole-proof generation and repair with large language models. In *ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023.
- [16] Huajian Xin, Daya Guo, Zhihong Shao, Zehui Ren, Qihao Zhu, Bo Liu, Chong Ruan, Wenda Li, and Xiaodan Liang. DeepSeek-Prover: Advancing theorem proving in LLMs through large-scale synthetic data. 2024.
- [17] DeepSeek-AI. DeepSeek-Prover-V1.5: Harnessing proof assistant feedback for reinforcement learning and Monte-Carlo tree search. 2024/2025 technical report.
- [18] Justin Asher. LeanExplore: A search engine for Lean 4 declarations. 2025. <https://arxiv.org/abs/2506.11085>. Code: <https://github.com/justincasher/lean-explore>.
- [19] Guoxiong Gao, Zeming Sun, Jiedong Jiang, Yutong Wang, Jingda Xu, Peihao Wu, Bryan Dai, and Bin Dong. LeanSearch v2: Global premise retrieval for Lean 4 theorem proving. 2026. <https://arxiv.org/abs/2605.13137>. Code: <https://github.com/frenzymath/LeanSearch-v2>.

- [20] FrenzyMath contributors. jixia: A static analysis tool for Lean 4. GitHub repository. <https://github.com/frenzymath/jixia>.
- [21] HunyuanProver team. HunyuanProver: A scalable data synthesis framework and guided tree search for automated theorem proving. 2025 technical report.
- [22] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of Thoughts: Deliberate problem solving with large language models. In *NeurIPS*, 2023. <https://github.com/princeton-nlp/tree-of-thought-llm>.
- [23] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language Agent Tree Search unifies reasoning, acting, and planning in language models. In *ICML*, 2024. <https://github.com/lapisrocks/LanguageAgentTreeSearch>.
- [24] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. miniF2F: A cross-system benchmark for formal Olympiad-level mathematics. In *International Conference on Learning Representations*, 2022.
- [25] Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics. 2023.